

P0332r1 : Relaxed Incomplete Multidimensional Array Type Declaration

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0332r1
Date: 2017-10-11
Reply-to: hcedwar@sandia.gov, balelbach@lbl.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Bryce Lebach
Contact: balelbach@lbl.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Athanasios Iliopoulos
Contact: athanasios.iliopoulos@nrl.navy.mil
Author: John Michopoulos
Contact: john.michopoulos@nrl.navy.mil
Audience: Library Evolution Working Group (LEWG)
Audience: Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0332.rst

Revision History	
P0009r0	Original multidimensional array reference paper with motivation, specification, and examples.
P0009r1	Revised with renaming from <code>view</code> to <code>array_ref</code> and allow unbounded rank through variadic arguments.
P0332r0	Relaxed array declaration moved from P0009.
P0332r1 (<i>current</i>)	Align with updates to P0009. Add examples of precedents.
References	
P0454r0	Wording for a Minimal <code>mdspan</code>
P0009r2	Multidimensional array reference specification
P0331	Multidimensional array reference motivation and examples

Motivation

The dimensions of multidimensional array reference `mdspan` (P0009) are declared a syntactically verbose property argument. We propose a trivial, non-breaking relaxation of the array type declaration in **11.3.4p3** to allow a concise and intuitive syntax for multidimensional declarations.

```
template< typename DataType , typename Properties... >
struct mdspan ;

// Three dimensional tensor type declaration with verbose syntax

using tensor = std::mdspan<double, std::extents<std::dynamic_extent, std::dynamic_extent, std::dynamic_extent>> ;

// Three dimensional tensor type declaration with concise syntax

using tensor = std::mdspan<double[] [] []> ;
```

This paper (P0332) is submitted to EWG when P0009 advances from LEWG to LWG.

Proposal

The current array type declarator constraints are defined in in **11.3.4.p3** as follows.

When several “array of” specifications are adjacent, a multidimensional array type is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted. In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (11.3.5). An array bound may also be omitted when the declarator is followed by an initializer (11.6) or when a declarator for a static data member is followed by a brace-or-equal-initializer (12.2). In both cases the bound is calculated from the number of initial elements (say, N) supplied (11.6.1), and the type of the identifier of D is “array of N T”. Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

For `mdspan` to have a concise and intuitive syntax the array type declarator constraints must be relaxed. The following wording change is proposed.

When several “array of” specifications are adjacent, a multidimensional array type is created. In declarations in which an incomplete object type is allowed any of the constant expressions that specify the bounds of the arrays may be omitted. The first of the constant expressions that specify the bounds of the arrays may be omitted

- in some cases in the declaration of a function parameter (11.3.5),
- when the declarator is followed by an initializer (11.6),
- when a declarator for a static data member is followed by a brace-or-equal-initializer (12.2), or
- if there is a preceding declaration of the entity in the same scope in which the bound was specified.

In the initializer cases the bound is calculated from the number of initial elements (say, N) supplied (11.6.1), and the type of the identifier of D is “array of N T”. In the preceding declaration case an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

Precedence and Feasibility

An incomplete array type `T[]` to concisely indicate an array of runtime length is used by `std::unique_ptr<T[]>` (23.11.1.3), `std::shared_ptr<T>` where `T` is `U[]` (23.11.2.2), and P0674 `make_shared<T[][N1][N2]>`.

This minor language specification change has been implemented with a trivial (one line) patch to Clang and was permissible in gcc prior to version 5.